

NOTACION BIG-O

Guia Completa de Complejidad Temporal

Que es Big-O?

La notacion Big-O expresa como el tiempo de ejecucion o la memoria de una operacion crece relativo al tamaño de la entrada (n).

Describe el PEOR CASO de rendimiento.

NOTACION	NOMBRE	EJEMPLO
$O(1)$	Constante	Acceder a elemento por indice
$O(\log n)$	Logaritmica	Busqueda binaria
$O(n)$	Lineal	Busqueda secuencial
$O(n \log n)$	Lineal-logaritmica	Merge sort
$O(n^2)$	Cuadratica	Bucles anidados

$O(1)$ - CONSTANTE

Tiempo siempre igual sin importar el tamaño de entrada.
El mas eficiente.

Ejemplos:

- lista[0] (Acceso directo)
- len(lista) (Obtener largo)
- diccionario['clave'] (Hash map)

$O(\log n)$ - LOGARITMICA

El tiempo crece logarítmicamente. Muy eficiente incluso con datos grandes.

Ejemplos:

- Busqueda binaria en lista ordenada
- Operaciones en arboles balanceados
- Con 1M elementos aprox. 20 operaciones

$O(n)$ - LINEAL

El tiempo crece proporcionalmente al tamaño de entrada.

Ejemplos:

- for x in lista: print(x)
- Encontrar maximo/minimo
- Busqueda en lista sin ordenar

$O(n \log n)$ - LINEAL-LOGARITMICA

Tipico en ordenamientos eficientes.

Ejemplos:

- Merge Sort
- Quick Sort (promedio)
- Con 1M elementos aprox. 20M operaciones

$O(n^2)$ - CUADRATICA

El tiempo crece exponencialmente con n .
MUY INEFICIENTE.

Ejemplos:

- Bubble sort
- Bucles anidados
- Con 1M elementos 1 BILLON de operaciones!

EJEMPLOS DE CODIGO

Ejemplo O(1) - Acceso Constante

```
lista = [1, 2, 3, 4, 5]
print(lista[0]) # O(1)
diccionario['clave'] = valor # O(1)
```

Ejemplo O(n) - Búsqueda Lineal

```
def buscar(lista, objetivo):
    for elemento in lista:
        if elemento == objetivo:
            return True
    return False
```

Ejemplo O(n^2) - EVITAR

```
def comparar_pares(lista):
    for i in lista:
        for j in lista:
            print(i, j)
```

Comparativa de Rendimiento

n=1000	O(1)	O(logn)	O(n)	O(nlogn)	O(n^2)
Ops	1	~10	1,000	~10,000	1,000,000

REGLA DE ORO: Elige O(1) u O(log n) cuando sea posible.

Evita O(n^2) especialmente con datos grandes.
La diferencia de rendimiento es ENORME.

CONCLUSION Y RECOMENDACIONES

1. PRIORIZA SIEMPRE $O(1)$:

Usa acceso directo cuando sea posible (arrays con indice, hash maps).

2. BUSQUEDA BINARIA ($O(\log n)$):

Si tienes datos ordenados, busqueda binaria es excelente.

3. OPERACIONES LINEALES ($O(n)$):

Aceptable para procesar todos los datos una sola vez.

4. EVITA $O(n^2)$:

Bucles anidados son mala idea con datos grandes.

Usa algoritmos mas eficientes (merge sort, quick sort).

5. RECUERDA:

El tamaño de entrada IMPORTA:

- 10 elementos: casi no hay diferencia
- 1,000 elementos: diferencia notable
- 1,000,000 elementos: $O(n^2)$ es IMPOSIBLE